

Comment l'alignement de
l'architecture logicielle sur les
contraintes matérielles et le tuning
de l'environnement d'exécution
permettent-ils de transformer la
Java Virtual Machine en une
plateforme déterministe et
distribuée pour le trading haute
fréquence ? Victorien GBAGUIDI

Organisme d'accueil : Société Générale CIB
Tuteur Pédagogique : Khalid BELHAJJAME
Maître d'apprentissage : Olivier CHANTEFOIN

Table des matières

Thanks	1
Acronyms	1
1 État de l'art	2
1.1 La physique de l'informatique appliquée à la programmation à faible latence	2
1.1.1 Les limites matérielles et les cycles CPU	2
1.1.2 Hiérarchie des caches et mécanique d'accès	3
1.1.3 Les opérations en cache et la "Cache Line" - Section 3.2, Drepper	3
1.1.4 Optimisation de l'usage du cache - Section 6.2, Drepper	3
1.1.5 Mesurer la performance - Section 1.8 Hennessy & Patterson	4
1.2 La JVM face au déterminisme financier	5
1.2.1 L'illusion du temps continu : Stop-The-World et SafePoints	5
1.2.2 La non-linéarité des performances : JIT et Warm-up	6
1.2.3 L'obstacle de l'abstraction mémoire et du layout	6
1.3 Algorithmes de Matching et Structures de Données "Cache-Friendly"	7
1.3.1 L'échec de la synchronisation standard : Le coût des verrous	7
1.3.2 Anatomie du Ring Buffer et du motif "Lock-Free"	7
1.3.3 Optimisation de la localité : Padding et Data-Oriented Design	8
1.4 L'OS et le Réseau : Les obstacles à la distribution	8
1.4.1 L'imprévisibilité de l'ordonnanceur Linux	8
1.4.2 Du Kernel Stack au Kernel Bypass : Vers le Zero-Copy réseau	9
1.4.3 Optimisation des messages : Simple Binary Encoding (SBE)	9
1.5 Analyse Comparative des Solutions Industrielles	10
1.5.1 Agrona et la gestion de la mémoire "Off-Heap"	10
1.5.2 Chronicle Software : Persistance et Communication Inter-Processus (IPC)	10
1.5.3 QuestDB et l'organisation orientée colonnes	11
1.5.4 Synthèse vers une approche "Zero-GC / Bit-Packed"	11
2 Structure de la Solution et Implémentation Itérative	11
2.1 Jalon 0 : La Baseline Java et son impact sur la latence	11
2.1.1 Architecture logicielle et Modélisation Objet	12
2.1.2 Analyse de la pression sur le Garbage Collector	12
2.1.3 Mesures initiales et identification des goulots d'étranglement	13
2.2 Jalon 1 : Le Séquenceur LMAX et l'ingestion ordonnée	13
2.2.1 Architecture du RingBuffer : Un cercle de données pré-allouées	13
2.2.2 Le modèle "Single-Writer"	14

2.2.3	Analyse comparative des performances Jalon 0 contre Jalon 1	14
2.3	Jalon 2 : Le Flyweight Pattern : Vers une empreinte mémoire constante	15
2.3.1	Élimination du False Sharing par le Padding	15
2.3.2	Analyse de l'efficacité du pipeline d'instructions	16
2.4	Jalon 3 : Communication Basse Latence avec Aeron (UDP)	16
2.4.1	Le paradigme de l'UDP fiable et l'évitement du TCP	16
2.4.2	L'architecture "Zero-Copy" via les Log Buffers	17
2.4.3	Configuration et Tuning pour le Matching Engine	17
2.5	Jalon 4 : Sérialisation SBE et Simulation de Cluster	18
2.5.1	La philosophie SBE : Le "Direct-to-Wire" sans parsing	18
2.5.2	Design du Schéma et Alignement Mémoire	18
2.5.3	Simulation de Cluster sous Docker-Compose	19
2.6	Jalon 5 : Analyse de la Latence de bout en bout (End-to-End) et Déterminisme	19
2.6.1	Mesurer l'invisible : La capture du "Wire-to-Matching"	19
2.6.2	Analyse de la gigue (Jitter) et des percentiles extrêmes	20
2.7	Jalon 5 : Limit Order Book en Data-Oriented Design (DOD)	21
2.7.1	La fin de l'indirection : Du modèle AoS au modèle SoA	21
2.7.2	Densité et Vitesse : le Bit-Packing	21
2.7.3	Algorithmes "Branchless" et prédictibilité du pipeline	22
2.8	Jalon 6 : Persistance Zero-Copy via MappedByteBuffer	23
2.8.1	Le mécanisme du Memory Mapping (mmap)	23
2.8.2	Architecture "Append-Only" et Sérialisation SBE	23
2.8.3	Optimisation du Flush et Durabilité	24
2.9	Jalon 7 : Métrologie avec JMH et HdrHistogram	24
2.9.1	L'usage du Java Microbenchmark Harness (JMH)	25
2.9.2	Analyse de la distribution via HdrHistogram	25
2.9.3	Interprétation des résultats : Le coût réel de l'abstraction	25
2.9.4	Métrologie de bout en bout : Du paquet réseau à l'exécution	26
2.10	Jalon 8 : Tuning du Noyau Linux et Isolation Système	26
2.10.1	Isolation des cœurs et CPU Pinning (isolcpus)	27
2.10.2	Gestion des interruptions matérielles (IRQ Affinity)	27
2.10.3	Optimisation de la gestion d'énergie et des "Huge Pages"	27
2.11	Jalon 9 : Analyse des Compteurs Matériels (Performance Monitoring Counters - PMC)	28
2.11.1	Quantification de la Localité de Cache (L1-dcache-misses)	28
2.11.2	Efficiency du Pipeline d'Instructions (Instructions Per Cycle - IPC)	28
2.11.3	Validation de l'Isolation de Cache et de l'Alignement (False Sharing)	29

Bibliography

29

Introduction

Dans l'écosystème financier contemporain, la vitesse n'est plus un simple avantage compétitif mais la condition essentielle de l'existence des acteurs de marché. Cette course effrénée à la vitesse a propulsé le trading haute fréquence au devant de la scène. Les décisions de vente et d'achat se prennent alors en quelques nanosecondes. Un élément crucial des systèmes financiers modernes est le **Order Matching Engine** qui fait correspondre des ordres d'achat à des ordres de vente pour exécuter une transaction. Une priorité selon le prix et le temps est alors effectuée. L'ordre d'achat le plus élevé pour un ordre de vente donné est priorisé, et à prix égal, l'ordre ayant été effectué en premier a la priorité. On voit alors de manière évidente les enjeux liés à la vitesse d'exécution.

Dans ce cadre, le C++ s'est imposé comme langage de programmation de référence pour ses possibilités d'optimisation bas niveau (gestion manuelle de la mémoire et interaction directe avec le matériel). À l'inverse, Java a mis du temps à trouver sa place au sein de l'univers de l'ultra faible latence. Son modèle de gestion automatique de la mémoire pilotée par le garbage collector crée des pauses imprévisibles qui augmentent fortement la latence dans les pires cas, une situation inacceptable pour les marchés financiers.

Une nouvelle philosophie propulsée par Martin Thompson, co-fondateur de LMAX, la mechanical sympathy propose que les performances d'un programme dépendent de la capacité du programmeur à le développer en s'alignant sur les contraintes matérielles du CPU et de la mémoire. Je vais dans ce mémoire explorer comment, en appliquant les concepts de mechanical sympathy, on peut transformer la JVM en environnement déterministe et à faible latence pour tirer partie du riche écosystème de Java tout en rivalisant avec les performances du C++ au niveau de la latence. La problématique sera donc : **Comment l'alignement de l'architecture logicielle sur les contraintes matérielles et le tuning de l'environnement d'exécution permettent-ils de transformer la Java Virtual Machine en une plateforme déterministe et distribuée pour le trading haute fréquence ?**

Pour répondre à cette problématique, mon plan s'articulera en deux parties :

- Un état de l'art : j'analyserai la hiérarchie des caches CPU, l'architecture NUMA, les obstacles internes de la JVM (Just In Time, Safepoints, Stop-the-World, ...) et nous étudierons les solutions industrielles telles que LMAX Disruptor, Aeron et Chronicle Software
- Une implémentation incrémentale d'un order matching engine : je détaillerai la construction de ma solution à travers dix jalons qui montrent l'évolution de la performance des différentes versions du matching engine en partant d'une base qui s'appuie sur des principes de programmation orientée objet puis nous appliqueront une architecture Zéro Garbage Collector et Data Oriented ainsi que des optimisations systèmes qui permet-

tront de montrer les gains apportés par l'application de la mechanical sympathy à notre problématique.

Mon intérêt pour ce sujet me vient de ma passion pour la programmation et je suis fermement convaincu qu'un véritable ingénieur logiciel doit maîtriser les concepts doit comprendre les concepts de mechanical sympathy par pure passion pour son métier. Au-delà de mon amour personnel de la programmation, je pense que ce mémoire est intéressant sur le plan professionnel car aujourd'hui, les problématiques de performance des programmes informatiques restent importantes économiquement dans les environnements financiers notamment où des millions d'ordres sont passé en une seconde, ce qui représente des millions d'euro échangés.

1 État de l'art

1.1 La physique de l'informatique appliquée à la programmation à faible latence

Références :

- Ulrich Drepper (2007) - What Every Programmer Should Know About Memory.
- J. L. Hennessy, D. A. Patterson (2017). Computer Architecture : A Quantitative Approach
- Benjamin J. Evans Optimizing Java

1.1.1 Les limites matérielles et les cycles CPU

Structure et type de RAM - Section 2.1 Drepper Nous classerons la mémoire vive en deux types : les RAM statiques (SRAM) et les RAM dynamiques (DRAM). Nous expliquerons que la SRAM offre une meilleure vitesse d'accès à la donnée mais est peu dense (6 transistors par bit) ce qui entraîne un surcoût de production mais aussi d'espace dans la machine et de consommation énergétique. Pour régler ce problème, on a la DRAM qui est plus dense mais plus lente.

Nous insisterons sur le coût technique de la DRAM en montrant comment les cycles de rafraîchissement créent des fenêtres d'indisponibilité physique de la barette mémoire, introduisant une latence non-déterministe dès le niveau matériel.

L'architecture NUMA et la distance physique - Section 2.2.2, Drepper Au-delà du type de RAM utilisé, l'emplacement physique de la donnée est critique. Nous détaillerons l'architecture Non-Uniform Memory Access (NUMA)

qui est standard sur les serveurs modernes. Nous expliquerons comment la distance physique entre le processeur et la barette de RAM crée deux classes d'accès : local (rapide) et remote (lent car traversant le bus d'interconnexion).

Nous montrerons comment l'architecture NUMA est la source principale de la gigue : si le scheduler de l'OS déplace notre thread de matching d'un cœur à un autre, le coût d'accès à ses données peut doubler instantanément.

1.1.2 Hiérarchie des caches et mécanique d'accès

Les différents types de cache - Section 3.1, Drepper Après avoir exposé la latence de la RAM principale, nous présenterons la hiérarchie de mémoire cache (L1, L2, L3) conçue pour masquer cette lenteur. Nous commenterons un schéma d'architecture pour illustrer le coût exponentiel des accès : de quelques cycles pour le L1 à plusieurs centaines pour la RAM.

Nous détaillerons la spécialisation des caches de niveau 1, séparés en L1i (Instructions) et L1d (Données), en expliquant pourquoi cette architecture est cruciale pour éviter que le chargement du code ne ralentisse le chargement du carnet d'ordres.

1.1.3 Les opérations en cache et la "Cache Line" - Section 3.2, Drepper

Nous définirons ici le concept de ligne de cache (cache line), typiquement de 64 octets. Nous expliquerons que le CPU ne charge jamais un octet isolé mais toujours un bloc complet.

Nous présenterons les processus de Cache Miss (défaut de cache) et de Cache Eviction, en montrant comment un code mal structuré force le processeur à attendre inefficacement les données venant de la RAM.

1.1.4 Optimisation de l'usage du cache - Section 6.2, Drepper

Nous présenterons des méthodes d'optimisation tirées de Drepper, en nous concentrant sur l'accès séquentiel. Nous apporterons la preuve que l'organisation en structures de données contiguës (tableaux) permet d'activer le Hardware Prefetcher. Nous expliquerons comment ce composant matériel anticipe les besoins du programme en chargeant les données en cache avant même que l'instruction de lecture ne soit exécutée, réduisant drastiquement le temps de traitement total. Nous présenterons également des applications de Hennessy & Patterson pour limiter les cache miss (données absentes du cache qui force une recherche dans un cache de niveau inférieur voire dans la mémoire vive) :

- Tirer parti de la spatial locality (des instructions qui sont proches en mémoire tendent à être utilisées ensemble) pour créer de plus gros blocs

- contenant plus d'instructions et ainsi limiter les accès au cache. Cela réduit également le compulsory miss mais augmente le miss penalty
- Je présenterai 3 métriques à optimiser relativement au cache : le hit time, miss rate et miss penalty avec les techniques ci-dessous :
 - Augmentation de la taille des blocs (Cache Lines) : L'utilisation de blocs plus larges permet de réduire les compulsory misses en exploitant la localité spatiale. Dans notre moteur, cela justifie l'utilisation de structures de données contiguës (tableaux de primitifs) : charger un prix permet de charger "gratuitement" les prix suivants déjà présents dans la même ligne de cache.
 - Caches multi-niveaux (Multilevel Caches) : L'introduction de niveaux intermédiaires (L2, L3) réduit la miss penalty. Nous étudierons comment le dimensionnement de notre carnet d'ordres doit idéalement tenir dans le cache L2 ou L3 pour éviter une chute brutale des performances lors de l'accès à la RAM.
 - Associativité du cache : Le processeur utilise l'associativité pour éviter que deux adresses mémoire différentes ne se "battent" pour le même emplacement physique en cache (conflict misses). Nous aborderons l'impact du cache placement sur le déterminisme des calculs.
 - Priorité aux lectures (Read Priority) : Les processeurs modernes utilisent des tampons d'écriture (Write Buffers). Cela permet de traiter en priorité les lectures (critiques pour la décision de trading) avant de confirmer les écritures en mémoire. Nous verrons comment le mot-clé volatile en Java peut perturber ce mécanisme en forçant le vidage du tampon.
 - Évitement de la traduction d'adresse : Le cache utilise des index virtuels pour gagner du temps sur la traduction d'adresse par la MMU (Memory Management Unit).
 - Le compromis Taille vs Latence : Conformément aux observations de Hennessy & Patterson, un cache plus grand réduit les défauts de capacité mais augmente mécaniquement le temps d'accès (hit time). Cette dualité sera au cœur de notre choix de structures de données "compactes" (Bit-packing) pour rester dans les couches de cache les plus rapides.

1.1.5 Mesurer la performance - Section 1.8 Hennessy & Patterson

Dans cette partie, nous comparerons deux métriques fondamentales de la performance : le **throughput** (débit) qui désigne le volume d'opérations traitées par période de temps et le **temps de réponse** (latence) qui mesure le délai d'exécution d'une instruction isolée. Dans le cadre de ce mémoire, nous nous focaliserons sur la latence. En effet, la rentabilité d'un algorithme dépend de sa capacité à réagir à un signal de marché plus vite que ses concurrents.

Nous privilégierons le **Wall Clock Time** (temps réel écoulé) car il englobe l'intégralité des délais subis par le programme : accès mémoire, opérations d'en-

trée/sortie et la surcharge système (OS overhead). À l'inverse, le **CPU Time** sera considéré comme incomplet car il ignore les phases d'attente (comme les pauses du Garbage Collector ou les interruptions réseau) qui sont pourtant critique en production.

De plus, nous rejetterons la latence moyenne au profit de l'analyse des percentiles et plus particulièrement la latence P99.99 qui indique que 99.99% des programmes auront une latence inférieure à celle mesurée. Ce choix se justifie car, en finance, un seul pic de latence peut entraîner des pertes de plusieurs millions d'euros. Cette réalité est totalement masquée par la moyenne arithmétique. Pour qualifier notre progrès, nous mesurerons le gain en vitesse d'exécution entre chaque jalon.

Enfin pour respecter la reproductibilité de nos mesures (critère essentiel d'un benchmark selon Hennessy et Patterson), nous utiliserons le framework **Java Microbenchmark Harness (JMH)**. Cet outil, référence de l'industrie, permet d'isoler les biais de la JVM et garantit que les gains observés entre nos différents jalons sont statistiquement significatifs.

1.2 La JVM face au déterminisme financier

Cette section analyse les problèmes liés à la JVM en ce qui concerne le déterminisme de la latence et sa minimisation. Nous analyserons notamment les pauses Stop The World ou encore le pointer chasing

Références :

- Tene, G. (2015). Understanding Java Garbage Collection and Latency.
- Benjamin J. Evans Optimizing Java

1.2.1 L'illusion du temps continu : Stop-The-World et Safepoints

En s'appuyant sur les travaux de Gil Tene, nous démontrerons que le principal obstacle au trading basse latence n'est pas la durée moyenne d'exécution, mais les interruptions imprévisibles appelées "hoquets" (hiccups). Nous analyserons le mécanisme de Stop-The-World (STW) : une phase où la JVM suspend l'exécution de tous les threads applicatifs pour permettre au Garbage Collector (GC) de manipuler la mémoire en toute sécurité. Nous introduirons le concept de Coordinated Omission : un biais de mesure où les outils de monitoring classiques "omettent" de comptabiliser le temps passé en pause STW, masquant ainsi des pics de latence catastrophiques pour la gestion du risque financier.

Le mécanisme des Safepoints - Optimizing, Section 13.b.i Java Au-delà du GC, nous détaillerons le concept de Safepoint, "point de rendez-vous" imposé par la JVM. Nous expliquerons que pour suspendre l'application, la JVM doit amener chaque thread à un état stable. Nous analyserons le Time To Safepoint (TTSP) : le délai d'attente parfois long avant que le dernier thread ne s'arrête, créant une

gigue (jitter) indépendante de la logique métier. Nous montrerons comment des boucles mal optimisées ou des opérations d'E/S bloquantes peuvent retarder l'atteinte d'un Safepoint, paralysant l'ensemble du moteur de matching.

1.2.2 La non-linéarité des performances : JIT et Warm-up

La dynamique de la compilation Just-In-Time - Optimizing Java, Chapitre 4. Contrairement au C++, le code Java n'est pas binaire au démarrage. Nous étudierons l'architecture HotSpot et ses deux compilateurs (C1 et C2). Nous détaillerons le processus de profilage : la JVM interprète d'abord le bytecode, identifie les méthodes critiques (hotspots) et les compile en code machine optimisé durant l'exécution. Nous justifierons ainsi la nécessité absolue du Warm-up (phase de chauffe) en HFT : l'obligation d'injecter des milliers de messages fictifs pour "entraîner" le compilateur JIT avant l'ouverture réelle du marché, évitant ainsi que les premiers ordres de la journée ne subissent la lenteur de l'interprétation.

Le risque de Déoptimisation - Optimizing Java, Section 4.c.ii Nous aborderons un danger spécifique au trading : la déoptimisation. Nous expliquerons comment un changement soudain de régime de marché (ex : volatilité extrême modifiant les embranchements logiques) peut invalider les suppositions du compilateur. La JVM doit alors "dé-compiler" le code pour revenir à un état interprété avant de re-compiler, provoquant un pic de latence massif au moment précis où la réactivité est la plus cruciale.

1.2.3 L'obstacle de l'abstraction mémoire et du layout

L'overhead des objets et la barrière de l'indirection - Optimizing Java, Chapitre 11 Nous analyserons pourquoi la structure "tout-objet" de Java est inefficace pour la localité des caches étudiée en partie 1.1. Nous détaillerons le layout d'un objet Java (Object Header) : chaque instance consomme 12 à 16 octets de métadonnées inutiles au métier, augmentant la pression sur le cache L1. Nous démontrerons l'impact dévastateur des pointeurs (références) : dans une Array-List<Order>, les données sont éparpillées en mémoire, forçant le processeur à multiplier les accès RAM (Pointer Chasing) et rendant le Hardware Prefetcher inopérant.

L'Hypothèse Générationnelle et le GC - Optimizing Java, Section 6.c (p. 159-161) Enfin, nous exposerons l'incompatibilité entre le carnet d'ordres et le Garbage Collector. Nous expliquerons la Weak Generational Hypothesis : Java est conçu pour des objets qui meurent jeunes. Or, les ordres stockés dans un carnet d'ordres survivent longtemps, finissant par être déplacés dans la "Old Generation". Nous montrerons que le nettoyage de cette zone est beaucoup plus coûteux et génère des pauses STW plus longues, justifiant notre future transition vers une architecture Zero-Allocation où la mémoire est gérée manuellement hors du contrôle du GC.

1.3 Algorithmes de Matching et Structures de Données "Cache-Friendly"

Cette section explore la conception de structures de données capables de s'affranchir des goulots d'étranglement de la synchronisation Java standard pour exploiter pleinement le parallélisme matériel.

Références :

- Thompson, M., et al. (2011). Disruptor : High performance alternative to bounded queues.

1.3.1 L'échec de la synchronisation standard : Le coût des verrous

Analyse de la contention - LMAX Disruptor (Section : Complexity)

Nous démontrerons pourquoi les structures de données classiques de la JVM (telles que `ArrayBlockingQueue`) sont inadaptées à l'ultra-basse latence.

Le coût de l'arbitrage OS : Nous expliquerons que l'utilisation de verrous (locks ou `synchronized`) force le noyau Linux à intervenir pour arbitrer la concurrence. Ce processus déclenche des Context Switches (changements de contexte) coûteux, ruinant le déterminisme du temps de réponse.

Le goulot d'étranglement de l'unité centrale : En s'appuyant sur le concept de "Uniprocessor Bottleneck", nous montrerons qu'une file d'attente avec un verrou unique transforme un système multi-cœurs en un système séquentiel où chaque cœur attend la libération d'une ressource unique.

Method	Time (ms)
Single thread	300
Single thread with lock	10,000
Two threads with lock	224,000
Single thread with CAS	5,700
Two threads with CAS	30,000
Single thread with volatile write	4,700

TABLE 1 – Analyse comparative du coût de la synchronisation (en ms)

Source : LMAX Disruptor : High performance alternative to bounded queues for exchanging data between concurrent threads

1.3.2 Anatomie du Ring Buffer et du motif "Lock-Free"

Le principe du Single-Writer et des Memory Barriers - LMAX Disruptor Nous détaillerons l'architecture du Ring Buffer comme alternative aux files d'attente liées.

- La pré-allocation de mémoire : Nous justifierons l'utilisation d'un tableau circulaire de taille puissance de 2, permettant d'allouer tous les objets au

démarrage. Cette approche garantit un régime de Zero-Allocation et une indexation ultra-rapide par masquage binaire.

- Maîtrise de la cohérence de cache : Nous expliquerons comment le Disruptor remplace les verrous par des Memory Barriers (via le mot-clé volatile). En utilisant le protocole de cohérence de cache (MESI), nous montrerons comment les threads communiquent sans jamais bloquer l'exécution du CPU.

1.3.3 Optimisation de la localité : Padding et Data-Oriented Design

Élimination du "False Sharing" - LMAX Disruptor (Section : False Sharing) Nous aborderons la technique de Padding (remplissage) pour protéger l'intégrité des lignes de cache.

- Isolation des curseurs : Nous démontrerons que si le curseur du "Producteur" et celui du "Consommateur" résident sur la même ligne de cache de 64 octets, chaque écriture invalide le cache de l'autre cœur. Nous expliquerons comment l'ajout de variables "mortes" (padding de 56 octets) garantit une isolation physique et une accélération massive des échanges.

Du modèle Objet au Data-Oriented Design Enfin, nous introduirons la transition entre la programmation orientée objet classique et le design orienté données.

- Array of Structures (AoS) vs Structure of Arrays (SoA) : Nous comparerons le modèle classique (un tableau d'objets Order éparpillés) au modèle optimisé (des tableaux de primitifs long[] contigus).
- Impact sur le Prefetcher : Nous prouverons que seule la structure contiguë permet au Hardware Prefetcher du CPU d'anticiper les lectures, réduisant les cache misses lors du parcours du carnet d'ordres.

1.4 L'OS et le Réseau : Les obstacles à la distribution

Cette section analyse comment l'environnement d'exécution (le système d'exploitation) et les protocoles de communication peuvent anéantir les gains de performance obtenus au niveau du code si une stratégie d'isolation et d'optimisation binaire n'est pas rigoureusement appliquée.

Références :

- Systems Performance, Brendan Gregg (2020)
- FIX Trading Community (2019). Simple Binary Encoding Specification

1.4.1 L'imprévisibilité de l'ordonnanceur Linux

Impact du Scheduler et des interruptions - Systems Performance, Brendan Gregg (2020) Nous démontrerons que le système d'exploitation

est, par nature, une source de gigue (jitter) pour les applications de trading haute fréquence.

- Le coût du Context Switching : En s'appuyant sur les chapitres de Gregg, nous expliquerons comment l'ordonnanceur Linux peut interrompre notre thread de matching pour exécuter des tâches système. Ce mécanisme, bien qu'invisible en informatique standard, provoque des pics de latence P99.99 inacceptables en trading.
- La pollution par les Interruptions (IRQs) : Nous analyserons comment les interruptions matérielles (provenant du réseau ou du disque) viennent "voter" des cycles CPU à l'application. Nous introduirons ici la solution technique de l'Isolation de Cœurs (isolcpus), visant à dédier des processeurs exclusifs au moteur de trading pour supprimer ce "bruit système".

1.4.2 Du Kernel Stack au Kernel Bypass : Vers le Zero-Copy réseau

Les limites de la pile réseau standard - Brendan Gregg (2020) Nous détaillerons le trajet complexe d'un paquet réseau TCP/UDP standard pour montrer son inefficacité en haute fréquence.

- Le goulot d'étranglement du passage en Kernel Space : Nous expliquerons que chaque paquet reçu doit être copié du tampon de la carte réseau vers l'espace noyau, puis vers l'espace utilisateur (User Space). Ce cycle de copie répétitif sature la bande passante mémoire et augmente la latence.
- La révolution du Kernel Bypass : Nous présenterons les technologies permettant à l'application de lire directement les données sur la carte réseau (ex : Solarflare Onload, DPDK). Cette approche élimine l'intermédiaire de l'OS et permet d'atteindre des latences réseau déterministes et proches du matériel.

1.4.3 Optimisation des messages : Simple Binary Encoding (SBE)

Nous justifierons le passage d'un protocole textuel (comme le FIX standard ou le JSON) vers un protocole binaire dense.

- Le coût du parsing et de l'allocation : Nous démontrerons que transformer une chaîne de caractères en objet Java génère une pression inutile sur le Garbage Collector et consomme des cycles CPU précieux.
- La philosophie SBE (Copy-Free / Allocation-Free) : Nous expliquerons comment le protocole Simple Binary Encoding permet d'accéder aux données directement dans le tampon réseau. En utilisant un encodage à schéma fixe, SBE permet au CPU de lire les champs d'un message sans aucune étape intermédiaire de "décodage".
- Alignement mémoire et efficacité CPU : Nous montrerons comment SBE aligne les données sur des frontières de 8 octets, permettant au processeur de charger les informations en un seul cycle d'horloge, maximisant ainsi l'usage des registres et du cache L1.

1.5 Analyse Comparative des Solutions Industrielles

Cette section synthétise les stratégies de pointe adoptées par les leaders de l'industrie (LMAX, Chronicle, QuestDB) pour transformer les contraintes physiques et logicielles étudiées précédemment en avantages compétitifs.

Références :

- Lawrey, P. (2017). Chronicle Software : Low Latency Microservices
- Documentation technique d'Aeron Messaging

1.5.1 Agrona et la gestion de la mémoire "Off-Heap"

Nous analyserons comment la bibliothèque Agrona permet de s'affranchir des limites de la gestion mémoire standard de la JVM.

- Manipulation directe des buffers : Nous détaillerons l'usage des Direct-Buffer et UnsafeBuffer qui permettent de lire et d'écrire directement dans la mémoire native, hors du contrôle du Garbage Collector. Cette approche élimine totalement les pauses STW liées au nettoyage des objets temporaires.
- Collections primitives et évitement de l'Autoboxing : Nous montrerons comment Agrona remplace les collections Java standards par des structures (ex : Int2ObjectHashMap) conçues pour manipuler des types primitifs, supprimant ainsi l'overhead mémoire des objets wrappers (Integer, Long).

1.5.2 Chronicle Software : Persistance et Communication Inter-Processus (IPC)

La fin du paradigme réseau traditionnel - Lawrey (2017) Nous étudierons l'approche de Chronicle Software qui permet à deux processus Java de partager le même segment de RAM physique via le noyau pour s'affranchir de la latence au niveau des communications réseau.

- L'exploitation des Memory Mapped Files : Nous expliquerons le fonctionnement des fichiers mappés en mémoire (MappedByteBuffer). En permettant à deux processus Java de partager le même segment de RAM physique via le noyau, Chronicle atteint des latences de communication inférieures à la microseconde, surpassant largement les sockets TCP/UDP.
- Déterminisme et Auditabilité : Nous soulignerons l'avantage majeur de cette architecture : la persistance native. La persistance est assurée de manière transparente par le noyau via le mécanisme de Page Cache, permettant une écriture asynchrone sur disque sans bloquer le thread critique.

1.5.3 QuestDB et l'organisation orientée colonnes

Nous ferons le lien avec le Data-Oriented Design(DOD) à travers l'exemple du moteur de stockage QuestDB.

- Le stockage en colonnes (Column-based) : Nous expliquerons pourquoi regrouper les données par type (tous les prix ensemble, toutes les quantités ensemble) est supérieur au stockage par ligne pour le calcul intensif. Cette organisation maximise le taux de Cache Hit en permettant au CPU de traiter des vecteurs de données contigus.
- Alignement et Vectorisation : Nous montrerons comment cette structure facilite l'utilisation des instructions SIMD (Single Instruction, Multiple Data) du processeur, permettant de traiter plusieurs ordres en un seul cycle d'horloge.

1.5.4 Synthèse vers une approche "Zero-GC / Bit-Packed"

En conclusion de cet état de l'art, nous définirons l'approche hybride retenue pour notre projet :

- **La stratégie Zero-GC** : En combinant les DirectBuffers d'Agrona et les principes de pré-allocation du Disruptor, nous viserons une exécution où aucun objet n'est instancié sur le chemin critique. Cette approche élimine non seulement les pauses du Garbage Collector, mais supprime également la surcharge mémoire liée aux en-têtes d'objets Java (Object Headers).
- **L'optimisation par "Bit-Packing"** : Pour surpasser les structures de données classiques, nous introduirons le concept de compression binaire. En compactant les informations essentielles (ID, prix, quantité) au sein de types primitifs long via des opérations de décalage (bit-shifting), nous réduirons l'empreinte mémoire au minimum. Cette densité maximale vise à maintenir les niveaux de prix les plus actifs au sein du cache L1 du processeur, minimisant ainsi les cycles d'attente lors de l'accès à la mémoire vive.

2 Structure de la Solution et Implémentation Itérative

2.1 Jalon 0 : La Baseline Java et son impact sur la latence

Références

- Shipilëv, A. Java Object Layout (JOL)
- Documentation. OpenJDK

Ce premier jalon technique vise à construire une implémentation fonctionnelle du moteur de matching en respectant les standards de développement Java conventionnels (POO classique). L'objectif n'est pas la performance, mais la création d'un point de référence permettant de mesurer l'impact réel des abstractions de la JVM sur la latence.

2.1.1 Architecture logicielle et Modélisation Objet

Dans cette phase, nous privilégions la lisibilité et la maintenabilité du code, conformément aux principes de l'ingénierie logicielle classique.

- Modèle de données : Chaque ordre de bourse est représenté par une classe `Order` contenant des attributs standards (long id, double price, int quantity, Side side). Ces objets sont instanciés à chaque nouveau message reçu sur le réseau.
- Gestion du Carnet d'ordres (Limit Order Book) : Pour stocker les ordres, nous utilisons des collections issues du package `java.util`. Les ordres d'achat et de vente sont triés dans deux `TreeMap`, permettant une recherche en $O(\log(n))$ tout en maintenant l'ordre de priorité prix-temps.
- Mécanismes de synchronisation : Afin de garantir l'intégrité des données face à des accès concurrents (ingestion réseau vs moteur de matching), le carnet d'ordres est protégé par des blocs `synchronized` ou des `ReentrantLock`.

2.1.2 Analyse de la pression sur le Garbage Collector

L'un des enseignements majeurs de ce jalon est l'observation du cycle de vie des objets. Dans un environnement de trading à haute fréquence, le débit de messages peut atteindre plusieurs dizaines de milliers d'ordres par seconde.

- L'instanciation massive : Chaque ordre créé génère environ 32 à 48 octets sur le tas (Heap). À un rythme de 100 000 ordres/seconde, l'application alloue plusieurs mégaoctets par seconde de données à très courte durée de vie.
- Le phénomène de "Churn" : Cette allocation frénétique remplit quasi instantanément la Young Generation (Eden Space). En utilisant des outils comme `VisualVM` ou `JConsole`, nous documentons la fréquence des "Minor GC".
- Impact sur la latence : Bien que les Minor GC soient rapides, ils déclenchent des micro-pauses "Stop-The-World". Nous analyserons comment ces pauses, cumulées à la gestion des verrous, créent une instabilité chronique de la latence, même lorsque le processeur n'est pas à pleine charge.

2.1.3 Mesures initiales et identification des goulots d'étranglement

Avant d'entamer toute optimisation, nous soumettons cette baseline à un stress-test rigoureux via un injecteur de trafic local.

- Profil de latence : Les premières mesures devraient révéler une latence moyenne satisfaisante (ex : 500µs), mais une dégradation brutale dès le percentile P95. Le P99.99 peut s'envoler à plusieurs millisecondes, un délai rédhibitoire en HFT.
- Contention sur les verrous : L'utilisation de Lock introduit une "file d'attente" logicielle. Nous utiliserons des Thread Dumps pour montrer que le thread de matching passe une partie significative de son temps à l'état BLOCKED ou WAITING, attendant que le thread réseau libère l'accès au carnet.
- Localité des données : En examinant la structure des TreeMap, nous soulignerons que chaque entrée est un objet Node éparpillé dans la mémoire. Cette fragmentation empêche le CPU d'utiliser efficacement ses caches L1/L2, forçant de nombreux allers-retours vers la RAM (DRAM).

2.2 Jalon 1 : Le Séquenceur LMAX et l'ingestion ordonnée

Après avoir identifié les limites structurelles de la baseline (Jalon 0), ce deuxième jalon introduit le concept de RingBuffer inspiré du LMAX Disruptor. L'objectif est de supprimer la contention sur les verrous (locks) en organisant le flux de données de manière circulaire et séquentielle.

2.2.1 Architecture du RingBuffer : Un cercle de données pré-allouées

Contrairement aux files d'attente dynamiques (LinkedBlockingQueue), le RingBuffer est une structure de données à taille fixe, allouée au démarrage du système.

- Le principe de l'indexation circulaire : Nous utilisons un tableau d'objets (Order[]) dont la taille est une puissance de deux (ex : 1024, 4096). Cette astuce mathématique permet de calculer l'index dans le tableau via un simple masque binaire (sequence & (capacity - 1)), une opération beaucoup plus rapide qu'un modulo classique pour le processeur.
- Pré-allocation et recyclage : Plutôt que de détruire et recréer des objets, le RingBuffer est rempli d'objets vides dès l'initialisation. Lors de l'arrivée d'un ordre, nous ne faisons que mettre à jour les attributs d'un objet existant.
- Avantage matériel : Cette structure garantit une localité spatiale. Les données étant contiguës en mémoire, le Hardware Prefetcher du CPU peut anticiper les lectures, réduisant drastiquement les cache misses constatés en Jalon 0.

2.2.2 Le modèle "Single-Writer"

La gigue (jitter) en Java provient souvent de la lutte entre threads pour obtenir un verrou. Le Jalon 1, s'inspirant du Disruptor, résout ce problème en imposant qu'un seul thread ait le droit d'écrire à un endroit donné du buffer.

- Séparation des responsabilités :

Le Producer (Ingester) : Reçoit les paquets réseau et les dépose dans le RingBuffer.

Le Consumer (Matching Engine) : Lit les messages et exécute la logique de matching.

- Barrières de mémoire (Memory Barriers) : La synchronisation ne se fait plus par synchronized, mais par la mise à jour d'une séquence volatile. En utilisant le protocole de cohérence de cache MESI, le thread de matching "voit" instantanément la nouvelle séquence publiée par le thread réseau sans qu'aucun des deux ne soit mis en pause par l'OS.
- Suppression du Context Switching : Comme aucun thread n'est jamais bloqué en attente d'un moniteur, l'ordonnanceur Linux n'a pas besoin d'intervenir. Le thread de matching tourne en "busy-spin" (boucle infinie de lecture), garantissant une réactivité immédiate dès qu'un ordre apparaît.

2.2.3 Analyse comparative des performances Jalon 0 contre Jalon 1

L'implémentation du séquenceur devrait transformer radicalement le profil de latence de notre moteur.

- Réduction de la queue de latence : En supprimant les ReentrantLock, nous devrions éliminer les "pics" de plusieurs millisecondes. La distribution de latence devient beaucoup plus étroite (plus déterministe).
- Débit (Throughput) : Sans la surcharge liée à la gestion des verrous et à la création d'objets, le moteur est capable de traiter un volume d'ordres par seconde nettement supérieur.
- Observation du "Back-pressure" : Nous introduisons ici un mécanisme de contrôle : si le moteur de matching est trop lent, le RingBuffer se remplit. Le producteur doit alors attendre que de la place se libère. Cette "pression inverse" est essentielle pour éviter l'effondrement du système sous une charge extrême (flash crash).

Le passage au RingBuffer marque la fin de l'arbitrage logiciel. Cependant, un nouveau problème surgit : deux threads lisant et écrivant sur des séquences proches en mémoire peuvent se ralentir mutuellement au niveau du silicium. C'est ce que nous traiterons dans le Jalon 2 avec le False Sharing.

2.3 Jalon 2 : Le Flyweight Pattern : Vers une empreinte mémoire constante

Dans les jalons précédents, chaque message entraînait encore la manipulation d'objets Java, même si ceux-ci étaient pré-alloués dans le RingBuffer. Le Flyweight Pattern (poids-mouche) pousse la logique du "Zero-Allocation" à son paroxysme.

- Le décodeur unique : Au lieu de copier les données du buffer réseau vers un objet Order, nous créons un objet "Vue" unique. Cet objet ne contient pas de données, mais uniquement des méthodes d'accès (getters/setters) pointant vers un décalage (offset) dans un bloc de mémoire brute (DirectBuffer).
- Suppression de l'indirection : En éliminant les références d'objets intermédiaires, nous supprimons ce que les experts appellent le "Pointer Chasing". Le CPU n'a plus besoin de suivre plusieurs adresses mémoire pour trouver une valeur ; il lit directement la donnée là où elle a été déposée par la carte réseau.
- Impact sur le GC : Puisqu'aucun nouvel objet n'est manipulé lors du matching, la JVM ne détecte aucun changement d'état sur le tas (Heap). Le risque de déclenchement d'un cycle de nettoyage devient quasi nul en régime de croisière.

2.3.1 Élimination du False Sharing par le Padding

Le processeur manipule la mémoire par blocs de 64 octets (Cache Lines). Un problème physique survient lorsque deux variables indépendantes, utilisées par deux cœurs différents, se retrouvent par mégarde sur la même ligne de cache.

- Le conflit de cohérence (MESI) : Si le thread de réception met à jour le curseur tail et que le thread de matching lit le curseur head, et que ces deux variables sont côte à côte en mémoire, le CPU considère qu'il y a un conflit. Chaque écriture sur l'un force l'invalidation du cache de l'autre cœur, créant une latence artificielle massive.
- La technique du Padding : Pour isoler nos variables critiques, nous insérons des champs "morts" (des types long inutilisés) avant et après nos curseurs. L'objectif est de "remplir" la ligne de cache pour que chaque "véritable" variable occupe seule ses 64 octets.
- Vérification avec JOL (Java Object Layout) : Nous utilisons l'outil d'inspection d'OpenJDK pour vérifier l'alignement réel en mémoire. Nous apportons ainsi la preuve que nos structures sont physiquement isolées, garantissant que les cœurs CPU travaillent en parallèle sans se polluer mutuellement.

2.3.2 Analyse de l'efficacité du pipeline d'instructions

L'application combinée du Flyweight et du Padding devrait transformer l'application en un moteur déterministe.

- Stabilité du P99.99 : En supprimant le False Sharing, nous devrions observer une disparition des "micro-pics" de latence qui persistaient en Jalon 1. La variance du temps de réponse s'approche de celle d'un système temps réel dédié.
- Maximisation de l'IPC (Instructions Per Cycle) : Grâce à la réduction des conflits de cache, le processeur peut exécuter plus d'instructions par cycle d'horloge. Le "coût" du passage d'un ordre dans le système devrait descendre sous la barre des quelques centaines de cycles CPU.

À ce stade, nous avons optimisé la machine locale au maximum des capacités de la JVM. Cependant, un moteur de trading isolé ne peut fonctionner seul. Le défi suivant consiste à sortir de la mémoire partagée pour affronter la latence du réseau, ce qui nous amène au Chapitre 2 avec l'implémentation d'Aeron. L'efficacité d'un moteur de matching ne se mesure pas seulement à sa vitesse d'exécution interne, mais à sa capacité à ingérer des ordres provenant du réseau sans distorsion de latence. Ce chapitre traite de l'affranchissement des piles réseau standards au profit de protocoles "Direct-to-Wire".

Références

- Thompson, M. (2014). Aeron : Efficient Reliable UDP Unicast and Multicast Messaging.

2.4 Jalon 3 : Communication Basse Latence avec Aeron (UDP)

Le passage d'une exécution locale à une architecture distribuée introduit le risque majeur de "l'indéterminisme réseau". Pour franchir cette étape, nous remplaçons les sockets TCP standards par le protocole Aeron, conçu par Martin Thompson pour répondre aux exigences de débit et de latence de l'industrie financière.

2.4.1 Le paradigme de l'UDP fiable et l'évitement du TCP

Le protocole TCP, bien que robuste, souffre de mécanismes intrinsèquement hostiles à la basse latence, tels que l'algorithme de Nagle ou le contrôle de congestion agressif.

- Suppression du Head-of-Line Blocking : En utilisant l'UDP comme couche de transport, Aeron évite qu'un seul paquet perdu ne bloque l'intégralité du flux de données. Aeron reconstruit une couche de fiabilité (retransmission sélective) beaucoup plus légère et prévisible.

- Contrôle de flux non-bloquant : Contrairement au TCP qui force le "back-pressure" au niveau du noyau, Aeron permet de configurer des stratégies de gestion de perte de données. En HFT, il est souvent préférable de rater un message et de le rattraper via un mécanisme applicatif plutôt que de figer tout le système pendant 40ms.
- Fragmentation et Réassemblage : Nous détaillerons comment Aeron découpe les messages SBE en "MTU-sized frames" de manière transparente, garantissant que le matériel réseau (switches) traite les paquets sans délai de fragmentation CPU.

2.4.2 L'architecture "Zero-Copy" via les Log Buffers

L'innovation majeure d'Aeron réside dans son utilisation intensive de la mémoire partagée et des fichiers mappés (mmap), permettant de supprimer les copies mémoires inutiles.

- Le Media Driver : Nous implémentons un Media Driver séparé (souvent isolé sur son propre cœur CPU). Ce composant gère l'envoi et la réception des paquets UDP, tandis que notre moteur de matching ne fait que lire dans une zone mémoire partagée appelée Log Buffer.
- Publication et Souscription sans Syscalls : Lorsqu'un message est publié, il est écrit directement dans le buffer RAM. Le Media Driver l'envoie ensuite sur la carte réseau (NIC). Ce design "lock-free" permet d'échanger des messages entre la Gateway et le Matching Engine sans jamais effectuer d'appels système (syscalls) coûteux sur le chemin critique.
- Conduites de données (Conduits) : Nous analyserons la structure des répertoires d'Aeron où chaque canal est un fichier physique mappé en mémoire, offrant une latence de communication inter-processus (IPC) proche des limites théoriques du bus PCIe.

2.4.3 Configuration et Tuning pour le Matching Engine

L'efficacité d'Aeron dépend de son paramétrage fin. Nous documentons ici les choix techniques effectués pour notre cluster simulé :

- Définition des Term Buffers : Nous dimensionnons les Term Buffers (typiquement 16MB ou 64MB) pour absorber les rafales d'ordres lors de l'ouverture des marchés sans déclencher de rotations de fichiers trop fréquentes.
- Utilisation du Idle Strategy : Pour le thread de souscription (celui qui attend les ordres), nous optons pour une BusySpinIdleStrategy. Ce choix force le processeur à interroger la mémoire en continu, garantissant que le passage de la donnée du réseau vers le carnet d'ordres se fait en quelques nanosecondes, au prix d'une consommation CPU de 100
- Monitoring via AeronStat : Nous utilisons l'outil aeron-stat pour surveiller les "Backlog" et les "NAKs" (Negative Acknowledgments). Une

augmentation des NAKs est le signe d'une saturation de la pile réseau ou d'un mauvais alignement des interruptions matérielles.

L'intégration d'Aeron transforme notre réseau en un bus de données quasi-transparent. Cependant, transporter des données rapidement est inutile si le format de ces données impose un temps de décodage long. C'est ce qui justifie l'introduction du protocole SBE dans le Jalon 4.

2.5 Jalon 4 : Sérialisation SBE et Simulation de Cluster

La performance du transport réseau (Aeron) doit impérativement être couplée à une stratégie de sérialisation à latence ultra-faible. L'utilisation de formats textuels (JSON, XML) ou même de formats binaires à schéma variable (Protobuf) introduit des étapes de parsing et d'allocation d'objets incompatibles avec la microseconde. Nous implémentons ici le Simple Binary Encoding (SBE).

2.5.1 La philosophie SBE : Le "Direct-to-Wire" sans parsing

Le protocole SBE, standardisé par la FIX Trading Community, repose sur une approche radicalement différente de la sérialisation classique. Au lieu de "décoder" un message, l'application "survole" les données brutes.

- Accès par Offset fixe : Dans SBE, chaque champ (ex : Price, Quantity, OrderID) se trouve à une position immuable dans le message. Le compilateur SBE génère des "stubs" (décodeurs) qui calculent l'adresse mémoire exacte d'une donnée en un seul cycle CPU.
- Évitement de l'Allocation : Contrairement à un parser JSON qui créerait des objets String ou Map, le décodeur SBE est un objet Flyweight (vu au Jalon 2) qui pointe directement sur le DirectBuffer d'Aeron. Aucune mémoire n'est allouée sur le tas lors de la lecture d'un ordre, éliminant toute pression sur le Garbage Collector.
- Streaming Access : SBE est conçu pour une lecture séquentielle. Le processeur lit les octets dans l'ordre où ils arrivent, ce qui maximise l'efficacité du cache L1 et évite les sauts de mémoire imprévisibles.

2.5.2 Design du Schéma et Alignement Mémoire

La performance de SBE ne provient pas seulement de son code, mais de la structure physique du message défini dans le fichier XML de schéma.

- Alignement sur les mots machine (64-bit) : Nous concevons notre schéma pour que les types long (8 octets) commencent toujours sur un index multiple de 8. Si un champ est mal aligné, le CPU doit effectuer deux cycles de lecture pour une seule valeur ; l'alignement garantit une lecture atomique et instantanée.
- Gestion des types primitifs : Nous bannissons les types flottants (double) au profit de types entiers (long) avec un multiplicateur fixe (ex : prix

stocké en "cents" ou "pips"). Cela évite les erreurs d'arrondi et permet au moteur de matching d'effectuer des comparaisons arithmétiques entières, beaucoup plus rapides au niveau matériel.

- Messages à taille fixe : Bien que SBE supporte les champs de longueur variable, nous privilégions des messages de taille fixe pour nos ordres de bourse. Cela permet au moteur de matching de savoir exactement où commence le message suivant dans le RingBuffer sans calcul complexe.

2.5.3 Simulation de Cluster sous Docker-Compose

Pour valider l'interopérabilité entre la Gateway et le Matching Engine, nous créons un environnement de test reproductible simulant les contraintes d'un centre de données (Datacenter).

- Isolation des Stacks Réseau : Via Docker-Compose, nous définissons des réseaux virtuels distincts. Chaque conteneur possède sa propre interface eth0 et sa propre pile TCP/IP-UDP. Cela nous permet de vérifier que le Media Driver d'Aeron gère correctement la résolution d'adresses et la retransmission de paquets.
- Contraintes de ressources (CPU Pinning) : Nous utilisons les options cpuset de Docker pour forcer le conteneur du Matching Engine sur un cœur CPU spécifique. Cela simule l'isolation de cœur (isolcpus) que nous déploierons sur le serveur de production au Chapitre 4.
- Validation du flux binaire : Grâce à des outils comme Wireshark (avec le plugin SBE), nous analysons les trames circulant entre les conteneurs. Nous vérifions que l'empreinte réseau est minimale et que les messages sont effectivement compacts, sans octets de métadonnées superflus.

L'alliance d'Aeron et de SBE permet de transporter et de lire une intention de trading en un temps record, souvent inférieur à 2 microsecondes de bout en bout. La "porte d'entrée" du système est désormais optimisée. Le défi se déplace maintenant vers le Chapitre 3 : comment traiter ces messages à l'intérieur du carnet d'ordres sans perdre cet avantage de vitesse ?

2.6 Jalon 5 : Analyse de la Latence de bout en bout (End-to-End) et Déterminisme

L'intégration d'Aeron et de SBE permet de mesurer pour la première fois la performance systémique réelle. Ce jalon ne se contente pas de valider la vitesse brute ; il analyse la distribution statistique du temps de trajet d'un ordre, de son émission par la Gateway à sa réception par la logique de matching, en passant par la pile réseau virtualisée.

2.6.1 Mesurer l'invisible : La capture du "Wire-to-Matching"

Pour quantifier l'apport des protocoles basse latence, nous mettons en place une instrumentation non intrusive.

- Horodatage Nanoseconde (Epoch) : Chaque message SBE inclut un champ `sendingTime` capturé au moment précis où il est écrit dans le buffer d'Aeron sur la Gateway. À la réception, le moteur de matching capture le `arrivalTime`. La différence nous donne la latence de transit, incluant la sérialisation, le transport UDP et le passage par le Media Driver.
- Élimination du "Coordinated Omission" : En suivant les principes de Gil Tene, nous nous assurons que notre système de mesure ne s'arrête pas pendant les pauses système. Si la Gateway est bloquée, le temps d'attente est comptabilisé dans la latence du message suivant, révélant ainsi la véritable expérience de l'utilisateur final.
- Impact du "Zero-Copy" : Nous documentons la réduction drastique du temps passé dans le noyau Linux. Contrairement au Jalon 0 où chaque message déclenchait un appel système `read()`, Aeron permet ici au moteur de matching de "poller" la mémoire partagée, réduisant la latence de transit de 70% par rapport à une implémentation socket standard.

2.6.2 Analyse de la gigue (Jitter) et des percentiles extrêmes

Le succès de ce jalon se mesure à l'étroitesse de la courbe de distribution. Un système HFT n'est pas jugé sur sa moyenne, mais sur la prévisibilité de son pire cas (P99.99).

- Analyse de la "Queue" (Tail Latency) : Grâce à HdrHistogram, nous visualisons les "outliers". Nous isolons les causes de gigue restantes : interruptions logicielles du noyau Docker, partages de ressources CPU entre conteneurs, et rares évictions de cache.
- Stabilité sous charge : Nous soumettons le cluster à un test de "Burst" (rafale). Nous démontrons que grâce au Ring Buffer d'Aeron et à la sérialisation SBE, la latence reste stable même lorsque le débit passe de 1 000 à 100 000 messages par seconde. Le système ne subit pas de dégradation exponentielle, typique des architectures basées sur les verrous.
- Corrélation avec le JIT : Nous observons l'effet du "Warm-up". Les 10 000 premiers messages montrent une latence erratique (compilation C1/C2), puis la courbe s'aplatit brusquement, signalant que le code machine est optimisé et "chaud".

L'utilisation combinée d'Aeron (transport) et SBE (format) devrait garantir que l'information arrive au cœur du moteur de matching en moins de 5 microsecondes dans notre environnement simulé. Cependant, cette vitesse d'ingestion met en lumière un nouveau goulot d'étranglement : la structure interne du carnet d'ordres. Si le traitement logique (matching) prend plus de temps que la réception réseau, le Ring Buffer finira par saturer. C'est cette problématique de "vitesse de calcul interne" que nous aborderons en réinventant le stockage des ordres via le Data-Oriented Design. L'optimisation du transport réseau réalisée au chapitre précédent a déplacé le goulot d'étranglement vers la logique

interne du carnet d'ordres (Limit Order Book). Ce chapitre détaille la transition d'une modélisation orientée objet vers une architecture orientée données (Data-Oriented Design), visant une localité de cache absolue.

Références

- Documentation officielle de Java NIO (New I/O)
- Documentation `sun.misc.Unsafe`

2.7 Jalon 5 : Limit Order Book en Data-Oriented Design (DOD)

Le carnet d'ordres (Limit Order Book) constitue le cœur réactif du système. Dans les jalons précédents, malgré l'usage du `RingBuffer`, la structure interne du carnet restait dépendante d'une approche orientée objet (AoS - Array of Structures). Ce jalon marque une rupture fondamentale : nous déconstruisons l'objet `Order` pour adopter une architecture orientée données (Data-Oriented Design), alignée sur la hiérarchie des caches du CPU.

2.7.1 La fin de l'indirection : Du modèle AoS au modèle SoA

En Java standard, un carnet d'ordres est souvent une collection d'objets. Chaque objet `Order` possède sa propre adresse mémoire, forçant le CPU à "chasser des pointeurs" (Pointer Chasing) pour accéder aux données.

- Le problème de la fragmentation : Dans un modèle AoS, pour comparer les prix de dix ordres, le CPU doit charger dix segments de mémoire différents. Si ces segments ne sont pas contigus, le processeur subit des Cache Misses systématiques, tombant de la vitesse du cache L1 (1 ns) à celle de la DRAM (100 ns).
- La solution SoA (Structure of Arrays) : Nous réorganisons la mémoire en regroupant les données par type. Tous les prix sont stockés dans un tableau primitif `long[] prices`, et toutes les quantités dans un `long[] quantities`.
- Exploitation du Hardware Prefetcher : En rangeant les prix de manière contiguë, nous permettons au processeur de détecter un accès séquentiel. Le Prefetcher anticipe alors la lecture des prix suivants avant même que l'instruction ne soit exécutée, garantissant que la donnée est déjà présente dans le cache L1 au moment du matching.

2.7.2 Densité et Vitesse : le Bit-Packing

Pour maximiser l'efficacité de chaque ligne de cache (64 octets), nous cherchons à stocker le plus d'informations possible dans le moins d'espace possible. Nous utilisons pour cela le Bit-Packing.

- Compression sans perte : Un type `long` en Java occupe 64 bits. En trading,

un prix et une quantité n'ont souvent pas besoin de 64 bits chacun. Nous "packons" deux entiers de 32 bits dans un seul long.

Exemple : `long packedOrder = (price « 32) | (quantity & 0xFFFFFFFFFL);`

- Réduction de la bande passante mémoire : Cette technique divise par deux le volume de données transitant entre la RAM et le CPU. Puisque le processeur est capable de manipuler des registres de 64 bits de manière atomique, nous extrayons le prix ou la quantité via des masques binaires (bit-masking) en un seul cycle d'horloge.
- Évitement des types flottants : Le Bit-Packing nous impose de travailler avec des entiers (Fixed-point arithmetic). Cela supprime le coût de passage par l'unité de calcul flottant (FPU) et élimine les imprécisions d'arrondi critiques en finance.

2.7.3 Algorithmes "Branchless" et prédictibilité du pipeline

Le matching d'ordres repose sur des comparaisons (ex : `if (bidPrice >= askPrice)`). Cependant, les branchements conditionnels mal prédits par le CPU peuvent vider le pipeline d'exécution, causant une perte de 15 à 20 cycles.

- Élimination des sauts conditionnels : Nous réécrivons les fonctions critiques en utilisant des opérations logiques ou des opérateurs ternaires que le compilateur JIT peut transformer en instructions CMOV (Conditional Move).
- Déterminisme algorithmique : En rendant le chemin d'exécution identique, que l'ordre soit exécuté ou non, nous stabilisons le temps de traitement. Le prédicteur de branchement du CPU n'est plus "pollué" par la volatilité du marché, garantissant une latence constante même lors des pics d'activité.
- Vectorisation potentielle : Cette structure de données contiguë prépare le terrain pour l'utilisation des instructions SIMD (Single Instruction, Multiple Data). Dans les versions futures (via Project Panama), nous pourrions comparer quatre prix simultanément en une seule instruction processeur.

L'application du DOD et du Bit-Packing transforme le carnet d'ordres en une machine de calcul binaire pure. Le moteur de matching ne traite plus des "ordres", mais des "flux de bits" optimisés pour le silicium. Une fois cette performance de calcul atteinte, le dernier défi est de s'assurer que l'écriture des logs ne ralentit pas cette mécanique : c'est l'objet de la section sur la persistance Zero-copy.

2.8 Jalon 6 : Persistance Zero-Copy via MappedByteBuffer

Dans un système financier, la performance ne peut se faire au détriment de la résilience. Chaque exécution d'ordre doit être journalisée pour répondre aux exigences de conformité (MiFID II) et permettre une récupération après sinistre. Cependant, les appels système classiques d'écriture sur disque (`write()`) introduisent des pauses inacceptables. Le Jalon 6 résout ce dilemme par l'implémentation d'une persistance "native" via les fichiers mappés en mémoire.

2.8.1 Le mécanisme du Memory Mapping (`mmap`)

L'utilisation de `java.nio.MappedByteBuffer` permet de lier directement une portion de l'espace d'adressage virtuel de la JVM à un fichier physique sur le disque (SSD NVMe).

- Suppression des copies entre espaces : Dans une architecture standard, une donnée est copiée du "User Space" (JVM) vers le "Kernel Space" (OS) avant d'atteindre le disque. Avec `mmap`, cette copie intermédiaire disparaît. Écrire un log revient à écrire dans un tableau d'octets (`byte[]`) en RAM.
- Le rôle du Page Cache : C'est le noyau Linux qui prend la responsabilité de la persistance. Lorsqu'une page mémoire est modifiée ("dirty page"), le système d'exploitation la synchronise avec le support physique de manière asynchrone. Pour le moteur de matching, l'opération est quasi instantanée car elle s'effectue à la vitesse de la mémoire vive.
- Accès Off-Heap : Ces buffers résident en dehors du tas Java (Heap). Par conséquent, même si le journal de transactions atteint plusieurs gigaoctets, il n'ajoute aucune charge de travail au Garbage Collector, car la JVM ne "scanne" pas cette zone mémoire lors de ses cycles de nettoyage.

2.8.2 Architecture "Append-Only" et Sérialisation SBE

Pour maximiser le débit d'écriture et minimiser la fragmentation du SSD, nous adoptons une structure de journalisation purement séquentielle (Append-Only Log).

- Journalisation binaire dense : Nous réutilisons le protocole SBE (étudié au Jalon 4) pour la persistance. Chaque événement (Trade, Cancel, NewOrder) est écrit sous forme d'une trame binaire de taille fixe. Cela permet non seulement de gagner de l'espace, mais aussi de garantir que le disque effectue des écritures alignées sur ses blocs physiques.
- Récupération instantanée (Recovery) : En cas de crash, le redémarrage du système consiste simplement à "remapper" le fichier. Le moteur de matching peut alors relire le journal à la vitesse de la RAM pour reconstruire l'état exact du carnet d'ordres en quelques millisecondes.

- Évitement du verrouillage de fichier : Nous utilisons des techniques de "Memory Barriers" pour nous assurer que l'index d'écriture est mis à jour atomiquement. Cela permet à un processus tiers (par exemple, un moteur de reporting) de lire le journal en temps réel sans jamais bloquer le moteur de matching principal.

2.8.3 Optimisation du Flush et Durabilité

La contrepartie de la vitesse du mmap est le risque de perte de données en cas de coupure de courant si les pages ne sont pas encore "flushées" sur le disque.

- Stratégies de synchronisation : Nous analysons le compromis entre `msync(ASYNC)` et `msync(SYNC)`. En trading haute fréquence, on privilégie souvent la délégation au noyau (asynchrone) pour maintenir la latence, tout en s'appuyant sur une réplication réseau (Aeron) vers un second nœud pour la sécurité.
- Alignement sur les Huge Pages : Pour optimiser encore l'accès, nous configurons l'OS pour utiliser des Huge Pages (2MB au lieu de 4KB). Cela réduit la taille de la table des pages du noyau et accélère la traduction d'adresses virtuelles en adresses physiques, un gain de quelques nanosecondes crucial lors des rafales de transactions.

Nous avons démontré qu'il est possible de détourner la machine virtuelle Java de sa vocation originelle pour en faire un environnement de programmation système de haute performance. En fusionnant le Data-Oriented Design (DOD) et la persistance Zero-Copy, nous avons créé une symbiose où le code Java épouse la topologie du processeur au lieu de la subir.

L'implémentation de la persistance Zero-copy clôture la phase de développement du moteur de matching. Nous avons maintenant un système capable de recevoir (Aeron), de transformer (SBE), d'exécuter (DOD) et de sauvegarder (mmap) un ordre de bourse sans jamais quitter le domaine de la microseconde. Nous allons maintenant nous attacher à prouver ces résultats par une méthodologie de benchmark scientifique.

2.9 Jalon 7 : Métrologie avec JMH et HdrHistogram

La mesure de la latence dans un environnement managé comme la JVM est un défi complexe. Entre les optimisations dynamiques du compilateur JIT et les pauses imprévisibles du Garbage Collector, une mesure naïve (via `System.nanoTime()`) est souvent biaisée. Ce jalon implémente une méthodologie rigoureuse pour isoler la performance réelle du moteur de matching.

Références

- Tene, G. How NOT to Measure Latency.
- Shipilëv, A. Java Microbenchmark Harness (JMH)

2.9.1 L’usage du Java Microbenchmark Harness (JMH)

Pour obtenir des chiffres incontestables, nous utilisons JMH, l’outil de référence développé par l’équipe performance d’OpenJDK. Il nous permet de neutraliser les effets de bord classiques de la JVM.

- Le cycle de "Warm-up" (Chauffe) : Avant chaque mesure, nous exécutons 10 000 itérations "à blanc". Cela permet au compilateur JIT de profiler le code et de passer de l’interprété au code machine optimisé (C2). Sans cette étape, les mesures incluraient le temps de compilation, faussant totalement la moyenne.
- Prévention des optimisations agressives : JMH utilise des "Blackholes" pour consommer les résultats de nos calculs. Sans cela, le compilateur pourrait détecter que le résultat du matching n’est pas utilisé et supprimer purement et simplement notre code du binaire final (Dead Code Elimination).
- Isolation par Fork : Chaque test est lancé dans un nouveau processus OS. Cela garantit que la mémoire et les profils de branchement d’un jalon précédent ne viennent pas polluer les résultats du jalon suivant.

2.9.2 Analyse de la distribution via HdrHistogram

En trading haute fréquence, la latence moyenne est une métrique avec peu de valeur. Ce qui importe, c’est la constance. Nous utilisons HdrHistogram (High Dynamic Range) pour capturer la "queue de latence" (Tail Latency).

- Capture des Outliers (Valeurs aberrantes) : Alors que la moyenne peut se situer à 500 ns, un seul GC peut propulser un ordre à 50 000 ns. HdrHistogram enregistre chaque mesure avec une précision constante sur une échelle logarithmique, permettant de zoomer sur les percentiles extrêmes (P99, P99.9, P99.99).
- Éradication du "Coordinated Omission" : En suivant les travaux de Gil Tene, nous configurons nos tests pour qu’ils soient "ouverts". Si le système subit une pause de 1 ms, nous ne sautons pas les mesures pendant ce laps de temps ; nous enregistrons une latence de 1 ms pour tous les ordres qui auraient dû être traités, révélant ainsi la dégradation réelle perçue par le marché.
- Comparaison visuelle des spectres : Nous générons des graphiques de distribution où l’on observe la "compression" de la courbe entre le Jalon 0 et le Jalon 6. Là où la baseline devrait présenter une traîne infinie, la version optimisée montre une chute brutale après le P99, signe d’un déterminisme quasi-parfait.

2.9.3 Interprétation des résultats : Le coût réel de l’abstraction

L’analyse croisée de JMH et HdrHistogram nous permet de tirer des conclusions définitives sur nos choix architecturaux.

- Quantification du gain "Zero-GC" : Nous isolons précisément le moment où, dans la baseline, le passage d'un seuil d'allocation déclenche un Minor GC. Dans la version optimisée, nous démontrons que la courbe reste stable même après des millions d'ordres, prouvant l'efficacité de notre approche Off-Heap.
- Validation de la "Mechanical Sympathy" : En comparant les temps d'exécution, nous montrons que le passage au Data-Oriented Design (Jalon 5) a réduit la latence de base non pas de quelques pourcents, mais d'un facteur 10, validant l'hypothèse que l'attente mémoire est le principal frein à la performance en Java.
- Bilan du Speedup : Nous calculons le Speedup global selon la loi d'Am-dahl pour chaque jalon, offrant une vue synthétique des progrès réalisés depuis la version naïve.

2.9.4 Métrologie de bout en bout : Du paquet réseau à l'exécution

Au-delà de la performance intrinsèque de l'algorithme, nous devons valider l'efficacité de notre pile de communication distribuée (Aeron/SBE).

- Mesure "Tick-to-Trade" vs "Micro-benchmark" : Alors que JMH isole le code Java, nous effectuons des tests de charge en conditions réelles. Nous mesurons le délai entre la réception d'un *UDP Datagram* et l'émission de la confirmation d'exécution. Cela permet de valider que notre architecture *Zero-Copy* ne rajoute pas de surcharge lors du passage du réseau vers la mémoire vive.
- Analyse de l'impact du "Jitter" réseau : En environnement distribué, la latence est polluée par les files d'attente des commutateurs (switches) et les interruptions des cartes réseau (NIC). Nous utilisons des outils de capture de paquets (tcpdump/wireshark) pour corréler les timestamps matériels avec nos mesures applicatives.

Ce jalon de métrologie transforme nos intuitions en certitudes. Il prouve que la microseconde n'est pas atteinte par "chance" sur quelques exécutions, mais qu'elle est la performance nominale du système, garantie par une architecture qui respecte les cycles de la machine.

2.10 Jalon 8 : Tuning du Noyau Linux et Isolation Système

Même le code Java le plus optimisé reste à la merci des décisions de l'ordonnanceur (scheduler) du système d'exploitation. Dans un environnement standard, Linux interrompt régulièrement les processus pour gérer les tâches de fond, les mises à jour de l'horloge système ou les interruptions réseau. Le Jalon 8 applique une stratégie de "silence numérique" pour garantir que le moteur de matching dispose de 100% des ressources CPU, sans interruption.

2.10.1 Isolation des cœurs et CPU Pinning (isolcpus)

Par défaut, le noyau Linux distribue la charge de travail sur tous les cœurs disponibles. Pour le trading haute fréquence, cette versatilité est une source de gigue (jitter).

- Configuration du bootloader : Nous modifions les paramètres de démarrage de l'OS via GRUB en utilisant l'option `isolcpus`. Cela force Linux à ignorer certains cœurs physiques (par exemple les cœurs 2 et 3) lors de l'ordonnancement des processus généraux.
- Affinité des threads (CPU Affinity) : À l'aide de l'affinité logicielle, nous "épinglons" notre thread de matching sur l'un de ces cœurs isolés. Ce thread ne subit plus de Context Switches involontaires ; il possède le cœur de manière exclusive, garantissant que le cache L1 reste "chaud" (rempli uniquement de nos données de trading) en permanence.
- Impact mesuré : Nous démontrerons que cette isolation réduit drastiquement les pics de latence P99.99, car le processeur n'est plus jamais forcé de sauvegarder l'état de notre moteur pour exécuter une tâche tierce.

2.10.2 Gestion des interruptions matérielles (IRQ Affinity)

Une carte réseau standard bombarde le CPU d'interruptions à chaque réception de paquet. Si cette interruption tombe sur le cœur qui exécute le matching, le pipeline du processeur est gelé pendant plusieurs microsecondes.

- Détournement des IRQ : Nous configurons le contrôleur d'interruptions (`/proc/irq/`) pour forcer le traitement des paquets réseau sur le cœur 0 (dédié au système).
- Séparation du trafic et du calcul : En séparant physiquement le cœur qui reçoit la donnée (I/O) du cœur qui la traite (Matching), nous créons un pipeline asynchrone parfait. Le moteur de matching "polle" la mémoire partagée sans jamais être interrompu par le matériel.

2.10.3 Optimisation de la gestion d'énergie et des "Huge Pages"

La latence est souvent victime des mécanismes d'économie d'énergie modernes qui mettent les cœurs au repos lors de micro-silences de quelques microsecondes.

- Désactivation des C-States : Nous forçons le gouverneur de fréquence CPU sur le mode performance. Cela empêche le processeur de baisser sa fréquence ou d'entrer en mode veille. Le passage d'un mode basse consommation à la pleine puissance peut prendre jusqu'à 20 μ s, ce qui est une éternité en HFT.
- Utilisation des Huge Pages (TLB Optimization) : Nous configurons le système pour utiliser des pages mémoire de 2 Mo au lieu de 4 Ko.

Réduction des Misses TLB : Cela réduit drastiquement la taille de la

table de pages que le CPU doit consulter. Moins de pages signifie que le Translation Lookaside Buffer (TLB) peut indexer une plus grande portion de notre carnet d'ordres, accélérant chaque accès mémoire de quelques nanosecondes critiques.

Ce jalon transforme le serveur en une appliance dédiée. En combinant un code Java "Zero-GC" avec un noyau Linux "silencieux", nous atteignons un état où la seule latence restante est la limite physique du silicium et de la propagation du signal.

2.11 Jalon 9 : Analyse des Compteurs Matériels (Performance Monitoring Counters - PMC)

La validation scientifique ne peut se contenter de mesures de latence externes (Jalon 7). Pour prouver l'efficacité de notre approche "Mechanical Sympathy", nous devons auditer le comportement du processeur durant l'exécution du matching. Ce jalon utilise les Performance Monitoring Counters (PMC), des registres matériels du CPU, via l'outil Linux perf, pour quantifier l'impact de nos optimisations au niveau micro-architectural.

2.11.1 Quantification de la Localité de Cache (L1-dcache-misses)

L'objectif central du Data-Oriented Design (Chapitre 3) était de maximiser le taux de succès du cache L1. Les compteurs matériels nous permettent de valider cette hypothèse.

- Mesure comparative : Nous comparons le nombre de L1-dcache-load-misses par opération de matching entre la Baseline (Jalon 0) et la version DOD (Jalon 6).
- Interprétation des résultats : Nous démontrerons une réduction massive, souvent supérieure à 90%, des défauts de cache L1. Cela prouve que notre structure en tableaux contigus (long[]) et l'utilisation du Hardware Prefetcher garantissent que la donnée est presque toujours présente dans le cache le plus rapide du CPU (1-4 cycles), évitant les allers-retours coûteux vers la DRAM (100-300 cycles).

2.11.2 Efficience du Pipeline d'Instructions (Instructions Per Cycle - IPC)

Un code performant n'est pas seulement un code qui attend moins la mémoire ; c'est un code qui maintient le pipeline d'exécution du CPU rempli d'instructions utiles.

- Le ratio IPC : Le processeur x86 moderne est capable d'exécuter plusieurs instructions par cycle d'horloge (super-scalabilité). Un IPC faible (ex : 0.5) indique que le CPU passe son temps à attendre (mémoire, branchements). Un IPC élevé (ex : 2.0+) indique une excellente fluidité.

- Impact du code "Branchless" : En analysant le compteur branch-misses, nous validerons que notre stratégie d'élimination des if/else (Section 3.1) a porté ses fruits. En réduisant les erreurs de prédiction de branchement, nous maintenons un IPC élevé et constant, garantissant que le moteur de matching traite les ordres à la vitesse nominale du processeur, sans "bulles" d'attente.

2.11.3 Validation de l'Isolation de Cache et de l'Alignement (False Sharing)

Enfin, nous utilisons les compteurs pour vérifier l'absence de conflits matériels invisibles au niveau logiciel.

- Audit du False Sharing : En surveillant les événements de cohérence de cache (comme `machine_clears.memory_ordering` sur Intel), nous apportons la preuve que notre technique de Padding (Jalon 2) a effectivement isolé physiquement nos curseurs critiques (head et tail).
- Zéro Invalidation : L'absence de ces événements confirme que les cœurs CPU du thread réseau et du thread de matching ne se "disputent" jamais la même ligne de cache de 64 octets, permettant une communication via la RAM à la vitesse maximale théorique du bus mémoire.

L'analyse des compteurs matériels clôture cette phase de démonstration. La corrélation directe entre l'efficacité du CPU (IPC élevé, chute des Cache Misses) et la stabilité de la latence P99.99 confirme la validité de notre approche. Nous avons apporté la preuve factuelle que la synergie entre une architecture logicielle Zero-GC, un protocole de transport à ultra-faible latence et une isolation système rigoureuse permet de surpasser l'implémentation Java standard. Cette performance n'est pas le fruit du hasard, mais le résultat d'un alignement délibéré du logiciel sur les contraintes micro-architecturales du matériel.